Binary Files in Java Programming

17 September 2026 20:07

What is a Binary File?

A binary file stores information in the form of bytes (zeros and ones) rather than readable characters.
The data is stored exactly as it exists in computers memory.
Unlike text files, binary files cannot be opened and understood using editors such as Notepad.

Characteristics of Binary Files

- Stores data in binary format.
- Faster than text files.
- Uses less storage space.
- Not human readable.
- Suitable for storing objects.
- Used in games, databases, multimedia applications, banking software, etc.

Advantages of Binary files

Faster processing: Since no conversion between characters and data types is required, reading and writing are faster.

Less Storage Space: Binary representation occupies less space than equivalent text.

Direct Object Storage: Java objects can be stored directly using serialization.

Better performance: Suitable for large applications where speed is important.

Disadvantages

- Cannot be edited manually.
- Requires Java programs to view the contents.
- Corruption is harder to detect visually.

Text file vs Binary Files

Feature	Text File	Binary File
Readable	Yes	No
Format	Characters	Bytes
Speed	Slower	Faster
Storage	Larger	Smaller

Manual Editing	Possible	Not Possible
Storing Objects	No	yes

Binary File Classes in Java
Java provides several classes for binary file handling.

Byte Stream Classes
These classes read and write raw bytes.

InputStream
^
|
FileInputStream
^
|
OutputStream
^
|
FileOutputStream

These are useful for:

- Images
- Videos
- Audio
- PDF files

Data stream classes
These classes read / write primitive data types.

DataInputStream
DataOutputStream

Methods include:

Writing methods:
writeInt()
writeDouble()
writeBoolean()
writeChar()
writeUTF() for String type

Reading methods:
readInt()
readDouble()
readBoolean()
readChar()
readUTF()

Object Stream Classes

These classes are used to store complete Java objects.

- ObjectInputStream
- ObjectOutputStream

These are the most commonly used classes for binary files.

Writing Primitive Data to a Binary File

Suppose we want to store:

- Roll Number
- Name
- Marks

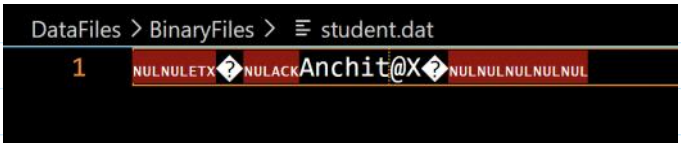
```
package BinaryFileHandling;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.util.Scanner;
public class WriteBinary {
    public static void main(String[] args) {
        String fileName, path = "DataFiles/BinaryFiles/";
        System.out.println("Enter file name: ");
        Scanner sc = new Scanner(System.in);
        fileName = sc.nextLine();
        try (FileOutputStream fio = new FileOutputStream(path+fileName,
true);
            DataOutputStream dos = new DataOutputStream(fio)) {
            System.out.println("Enter roll number: ");
            int rollNo = sc.nextInt();
            sc.nextLine();
            System.out.println("Enter name: ");
            String name = sc.nextLine();
            System.out.println("Enter marks: ");
            double marks = sc.nextDouble();
            dos.writeInt(rollNo);
            dos.writeUTF(name);
            dos.writeDouble(marks);
            System.out.println("Data written successfully.");
        } catch (Exception e) {
            // TODO: handle exception
        }
```

```
            finally{
                sc.close();
            }
        }
    }
```

Output:

```
Enter file name:
student.dat
Enter roll number:
1001
Enter name:
Anchit
Enter marks:
99
```

File:




```
package BinaryFileHandling;
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;
public class ReadBinaryData {
    public static void main(String[] args) {
        String fileName, path = "DataFiles/BinaryFiles/";
        System.out.println("Enter file name: ");
        Scanner sc = new Scanner(System.in);
        fileName = sc.nextLine();

        try (FileInputStream fis = new FileInputStream(path+fileName);
            DataInputStream dis = new DataInputStream(fis)){
            int roll = dis.readInt();
            String nm = dis.readUTF();
            double m = dis.readDouble();
            System.out.println("Roll number: " +roll);
            System.out.println("Name: " + nm);
            System.out.println("Marks: "  + m);
        } catch (FileNotFoundException e) {
```

```
            e.printStackTrace();
            System.err.println("File is not present in the given
folder.");
        } catch (IOException e1) {
            e1.printStackTrace();
            System.err.println("Network error");
        }
        finally{
            sc.close();
        }
    }
}
```

```
Output:
   Enter file name:
   student.dat
   Roll number: 1001
   Name: Anchit
   Marks: 99.0
```

Reading multiple records from the binary File

```
package BinaryFileHandling;
import java.io.DataInputStream;
import java.io.EOFException;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;
public class ReadBinaryData {
    public static void main(String[] args) {
        String fileName, path = "DataFiles/BinaryFiles/";
        System.out.println("Enter file name: ");
        Scanner sc = new Scanner(System.in);
        fileName = sc.nextLine();

        try (FileInputStream fis = new FileInputStream(path+fileName);
            DataInputStream dis = new DataInputStream(fis)){
                while (true) {
                    int roll = dis.readInt();
                    String nm = dis.readUTF();
                    double m = dis.readDouble();
                    System.out.println("Roll number: " +roll);
                    System.out.println("Name: " + nm);
                    System.out.println("Marks: "  + m);
                }
```

```
            }
            catch(EOFException eofe){
                System.out.println("============================");
                System.out.println("End of File.");
                System.out.println("============================");
            }
            catch (FileNotFoundException e) {

                e.printStackTrace();
                System.err.println("File is not present in the given
folder.");
            } catch (IOException e1) {
                e1.printStackTrace();
                System.err.println("Network error");
            }

            finally{
                sc.close();
            }
        }
}
```

Output:

```
   Enter file name:
   student.dat
   Roll number: 1001
   Name: Anchit
   Marks: 99.0
   Roll number: 1002
   Name: Aadit
   Marks: 90.0
   ============================
   End of File.
   ============================
```

Modifying or Deleting record from binary File

Modify: Locate the required record change its fields and write the modified record back.

Delete: Create a temporary binary file, copy all records except the one to be deleted, then replace the original file.

Java's **RandomAccessFile** is particularly useful when records have fixed size because it allow us to directly move to a record using **seek()**.

Roll Number ->int
Name -> fixed-size String
Marks -> double

Java classes Used

OK for this type of application we can use:

Class	Purpose
RandomAccessFile	Read, write and modify records at specific positions.
File	Check, delete and rename files.
IOException	Handle file-related errors

Homework:

Question 1 – Student Binary File Management

Write a Java program to create a binary file named student.dat inside the folder DataFiles/BinaryFiles/. The file should store student records containing:

- Roll Number – int
- Name – String
- Marks – double

The program should display the following menu:

1. Add a new student record
2. Display all student records
3. Search for a student by roll number
4. Display students scoring more than 75 marks
5. Exit

Requirements:

- Use FileOutputStream and DataOutputStream to write records.
- Use FileInputStream and DataInputStream to read records.
- Multiple records must be stored in the same binary file.
- Use EOFException to identify the end of the file while reading.
- When searching, compare the entered roll number with every record.
- Display an appropriate message if the student does not exist.
- Use try-with-resources wherever appropriate.
- Handle FileNotFoundException and IOException separately.

Sample records:

1001 Anchit 99.0
1002 Aadit 90.0
1003 Rohan 68.5
1004 Ananya 85.0

Sample menu:

```
============================
       STUDENT DATABASE
============================
1. Add Student
2. Display All Students
3. Search Student
4. Display Students Above 75
5. Exit
Enter your choice:
```

Challenge: Add another option to display the **average marks of all students** stored in the binary file.

Question 2 – Employee

Write a Java program to maintain an employee binary file named employee.dat. Each employee record contains:

- Employee ID – int
- Employee Name – fixed-size String
- Salary – double

The program should provide the following menu:

```
================================
       EMPLOYEE MANAGEMENT
================================
1. Add Employee
2. Display All Employees
3. Search Employee
4. Exit
```

For this program, assume that the employee name occupies **20 characters** in the binary file.

For example:

```
Employee ID : 101
Name        : Satyender
Salary      : 75000.0
```

Requirements:

1. Use DataOutputStream to initially create and store employee records.
2. Use DataInputStream to display all records.
3. Search for an employee using Employee ID.

Question 3

Write a Java program to manage a binary file named student.dat containing the following student information:

- Roll Number – int
- Name – String
- Marks in Mathematics – double
- Marks in Computer Science – double
- Marks in Physics – double

The program should provide the following menu:

```
==================================
       STUDENT RECORD SYSTEM
==================================
1. Add Record
2. Display All Records
3. Search by Roll Number
6. Exit
```

Part A – Add Records

Use:

FileOutputStream

DataOutputStream

to append student records to the binary file.

Part B – Display Records

Use:

FileInputStream

DataInputStream

EOFException

to read and display all records.

For every student, calculate and display:

Total Marks

Percentage

Part C – Search

Ask the user for a roll number and search the entire binary file.

Display the complete record if found.